

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы

Студент гр. 4382

Пермитин В. А.

Руководитель

Жангиров Т. Р.

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Пермитин В. А.

Группа: 4382

Тема практики: Генетические алгоритмы

Задание на практику:

Для заданного неориентированного графа $G = (V, E)$, необходимо найти множество вершин S , которое является вершинным покрытием графа и минимально.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 00.00.0000

Дата защиты отчета: 00.00.0000

Студент	_____	Пермитин В. А.
Руководитель	_____	Жангиров Т. Р.

АННОТАЦИЯ

Цель работы: познакомиться с генетическими алгоритмами для решения задач оптимизации и поиска

Языки программирования: Python3, C++, C#, Java, Golang, Javascript

Общее описание заданий: поиск аппроксимирующей функции, приближение функций, нахождение экстремумов, решение оптимизационных задач с использованием генетических алгоритмов (методичка будет выложена)

Бригады: от 1 до 3 человек. Две бригады могут взять одно и то же задание, если реализация будет на разных ЯП.

Требование к решению:

Программа должна иметь GUI.

Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.

Алгоритмы реализуются самостоятельно без использования готовых решений.

Настройкой параметров алгоритмов должна производиться пользователем.

Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключение между разными текущими решениями.

Возможность перейти к конечному решению пропустив отображение всех шагов.

Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

Доп. баллы даются за:

Реализацию возможности вернуться на несколько шагов назад.

Этапы выполнения задания:

- 1) Формирование бригады, выбор задания и ЯП, распределение ролей в бригаде
- 2) Демонстрация прототипа GUI и плана по решению задачи (описание формата данных, используемой функции качества, и.т.д.)
- 3) Промежуточная сдача программы (несколько итераций). На данном этапе должен быть частично-функционирующий GUI, работающий алгоритм, но возможно, требующий корректировок.
- 4) Демонстрация финальной версии программы и отчета.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	7
1.1 Предметная область	7
1.2 Описание формата данных.....	7
1.3 Задачи практики	7
2 РЕЗУЛЬТАТЫ РАБОТЫ	8
2.1. Виды кодирования.....	8
2.2. Подбор целевой функции генетического алгоритма.....	8
2.3 Выбор метода скрещивания	9
2.4 Выбор метода отбора	9
2.5 Выбор метода мутации	10
2.6 Реализация генетического алгоритма	10
2.7 Реализация графического интерфейса	11
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	13
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16

ВВЕДЕНИЕ

Вершинное покрытие – это такое подмножество вершин графа $G(V, E)$, что для любого ребра существует такая вершина из подмножества вершин, что эта вершина является концом ребра.

Задача минимального вершинного покрытия – это задача поиска минимального по включению вершинного покрытия.

Для решения этой задачи будет реализован генетический алгоритм.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы активно применяются в обучении с подкреплением, помогают подбирать параметры для моделей машинного обучения, помогают решать задачи без математического представления (например, поиск подходящих цветов)

1.2 Описание формата данных

Был реализован генетический алгоритм в классе GeneticAlgorithm. Его конструктор принимает на вход:

- 1) Входную матрицу смежности (причем необязательно симметричную, симметричность проверяется в GUI): List[List[float]]
- 2) Размер популяции: int
- 3) Штраф за нарушение жесткого ограничения: int
- 4) Вероятность скрещивания родителей: float
- 5) Вероятность мутации одного из родителей: float
- 6) Размер турнира: int
- 7) Количество элиты: int

Для запуска генетического алгоритма был реализован метод run. Он принимает на вход только один int – это количество итераций генетического алгоритма. На выходе метод выдает самую приспособленную особь и массив максимальной приспособленности на каждой итерации.

1.3 Задачи практики

Задачи:

- 1) Выбор метода кодирования индивидуума
- 2) Подбор целевой функции для генетического алгоритма
- 3) Выбор метода скрещивания
- 4) Выбор метода отбора
- 5) Выбор метода мутации
- 6) Реализация генетического алгоритма
- 7) Реализация графического интерфейса

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Виды кодирования

Были рассмотрены следующие виды кодирования:

- 1) Бинарная кодировка - решение представлено в виде массива 0 и 1
- 2) Порядковая кодировка - решение представлено в виде массива натуральных чисел, где учитывается их порядок
- 3) Вещественная кодировка - решение представлено в виде массива вещественных чисел. Целые числа частный случай.

Главная задача программы – найти наименьшее вершинное покрытие. То есть, нужно найти такое подмножество вершин, что хотя-бы один конец каждого ребра входит в это подмножество. Я решил представить подмножества вершин с помощью бинарной кодировки, где если ген равен 1, то вершина включена, если ген равен 0, то ген не в подмножестве.

Порядок выбора вершин не важен, а решение не нужно представлять в виде вещественных чисел, значит, нужно выбирать бинарную кодировку.

2.2. Подбор целевой функции генетического алгоритма

Генетический алгоритм ищет самых приспособленных индивидуумов, так что нужно максимизировать функцию приспособленности для тех индивидуумов, у которых вершинное покрытие имеет наименьшее количество вершин. Поэтому введу два ограничения:

- 1) Жесткое ограничение – не должно быть ребер, оба конца которых не находятся в подмножестве
- 2) Еще одно ограничение – вершинное покрытие должно быть минимальным

Для выполнения этих ограничений были реализованы штрафы.

Пусть k – количество ребер, которые не покрываются текущим решением, p – количество вершин. А n – количество вершин в покрытии. Тогда целевая функция будет следующая: $p - n - k \cdot \text{ШТРАФ}$, где

ШТРАФ (> 1) – размер штрафа за непокрытое ребро. ШТРАФ вводится для того, чтобы решение с наибольшим количеством непокрытых ребер точно отверглось.

2.3 Выбор метода скрещивания

Для скрещивания двух родителей я реализовал равномерное скрещивания.

Я ознакомился со следующими видами скрещиваний:

- 1) Одноточечное скрещивание
- 2) Двухточечное скрещивание
- 3) Равномерное скрещивание
- 4) Упорядоченное скрещивание
- 5) Скрещивание смещением
- 6) Имитация двоичного скрещивания

Было выбрано равномерное скрещивание, потому что если в графе есть n вершин, то существует $n!$ способов переставить порядок этих вершин, при этом все эти графы будут изоморфны друг другу. Можно расставить вершины так, чтобы одноточечное или двухточечное скрещивание выбирало в основном выбирало такие точки скрещивания, что родители будут обмениваться минимальным количеством вершин из минимального вершинного покрытия. Значит, такой метод в таком случае будет сходиться дольше.

Было выбрано равномерное скрещивание, потому что такое скрещивание дает наибольшее разнообразие подмножеств вершин, что и нужно.

2.4 Выбор метода отбора

Я ознакомился со следующими видами отбора:

- 1) Правило рулетки
- 2) Стохастическая универсальная выборка
- 3) Ранжированный отбор
- 4) Масштабирование приспособленности

- 5) Турнирный отбор
- 6) Отбор усечением
- 7) Элитарный отбор
- 8) Отбор вытеснением

Мной был реализован элитарный отбор с добором через турнирный отбор.

Элитарный отбор больше всего подходит в этой задаче, потому что если было найдено решение с самой большой приспособленностью, то при его мутировании и скрещивании всегда можно найти решение с еще большей приспособленностью, если оно есть и алгоритм не застрял на нескольких немного неизменяющихся индивидуумах (преждевременная сходимость).

Турнирный отбор был выбран, потому что он помогает оставлять решения с самой большой приспособленностью в популяции, и при этом дает разнообразие, которое помогает избежать преждевременной сходимости.

2.5 Выбор метода мутации

Я ознакомился со следующими видами мутации:

- 1) Инвертирование бита
- 2) Мутация обменом
- 3) Мутация обращением
- 4) Мутация перетасовкой
- 5) Вещественная мутация

Я реализовал мутацию инвертированием бита, потому что данная мутация позволяет случайно включить или исключить вершину из множества, добавляя большее разнообразие, и тем самым перебирая возможные варианты решения.

2.6 Реализация генетического алгоритма

Генетический алгоритм был реализован в файле `genetic_algorithm.py`. Для вызова генетического алгоритма можно использовать методы `run` (полный

запуск с просчетом всех итераций) и `run_iteration` для просчета одной итерации по переданной популяции.

2.7 Реализация графического интерфейса

Для реализации графического интерфейса была использована библиотека Tkinter.

Весь интерфейс был разбит на четыре строчки. Управление положением каждой части идет через геометрический менеджер Grid.

Первая строка – кнопки для работы с матрицей смежности. Эти кнопки были помещены в виджет Frame, и для этого виджета также использовался геометрический менеджер Grid. Первые три кнопки расположены в строке 0, остальные три – в строке 1. Причем, эти строки не относятся к строкам родительского виджета, для виджета Frame была сделана отдельная таблица для кнопок. Чтобы кнопки прилипли к бокам, им был передан аргумент `sticky="ew"` для растягивания кнопок по ширине.

Вторая и третья строка – это два графика, и матрица смежности, которая занимает две строки (`rowspan=2`). Для матрицы был выделен виджет Frame, относительно родительского виджета Frame располагается на строке 1 и колонке 0. Для того, чтобы Frame прилипал ко всем своим краям, был передан аргумент `sticky="nsew"` (north, south, east, west – направления прилипания). Внутри этого Frame находится много виджетов Entry, эти виджеты отвечают за ввод текста. Для поддержания симметричности матрицы был использован метод `trace_add` для каждого хранимого в матрице значения. Если где-то в матрице обновляется значение по индексам i, j , то ячейка j, i также принимает это значение (только если ячейка j, i имеет другое с i, j значение, для избежания рекурсии).

В строке 1, колонке 1 и в строке 2, колонке 1 расположены два графика. Верхний график – это граф, построенный по матрице смежности. Нижний график – это график приспособленности в зависимости от итерации. У библиотеки `matplotlib` есть интеграция с `tkinter`, так что для добавления графиков можно использовать класс `FigureCanvasTkAgg`, который создаст за

меня виджет для графика. Для каждого графика был создан виджет Frame, внутри этого фрейма был использован геометрический менеджер Pack с аргументами `fill=tk.BOTH`, `expand=True`. Первый аргумент отвечает за растяжение по горизонтальной и вертикальной осям виджета Frame. Вторым – за растяжение Frame при увеличении окна. Наверное, можно было обойтись без лишнего виджета Frame, но он не мешает отрисовке интерфейса.

В строке 3 находится Frame, отвечающий за поля ввода параметров. Параметров для генетического алгоритма нужно немало, и для избежания повтора кода я вынес код для создания кнопок внутри Frame строки 3 в метод `add_parameter_button`. Этот метод создает внутри Frame кнопку и располагает ее в заданном месте с помощью Grid. На вход `add_parameter_button` передается строка, столбец, текст-подсказка, имя атрибута и дополнительные аргументы для геометрического менеджера. Строка и столбец позволяют расположить поле ввода внутри Frame-а, текст-подсказка нужен для понимания того, за что поле отвечает, имя атрибута нужно для задания нового атрибута внутри класса `GeneticAlgorithmApp`, иначе не получится обращаться к новому полю в дальнейшем и читать из него данные.

И последняя строка 4 содержит Frame, в котором находится 4 кнопки для управления генетическим алгоритмом, и одна надпись для отображения текущего шага итерации.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Для решения данной задачи были использованы следующие технологии:

- 1) Python
- 2) Tkinter
- 3) Matplotlib
- 4) Networkx

Для установки необходимых библиотек можно использовать requirements.txt и пакетный менеджер pip, встроенный в Python.

Исходный код программы находится в [1].

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики были изучены принципы работы генетических алгоритмов и их применение для решения задач комбинаторной оптимизации. Основной практической частью стала разработка программного средства для поиска минимального вершинного покрытия неориентированного графа.

В рамках работы был спроектирован и реализован генетический алгоритм, включающий бинарное кодирование особей, равномерное скрещивание, элитарно-турнирный отбор и мутацию инвертированием бита. Для оценки качества решений была разработана целевая функция с системой штрафных коэффициентов за нарушение жестких ограничений.

Также в ходе выполнения задания были освоены навыки разработки интерактивных графических интерфейсов на языке Python. С использованием библиотек Tkinter, Matplotlib и Networkx создано полноценное приложение, позволяющее задавать граф через матрицу смежности, гибко настраивать параметры алгоритма и в реальном времени визуализировать процесс эволюции популяции, а также отслеживать динамику изменения функции приспособленности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Исходный код программы для решения задачи поиска минимального вершинного покрытия [Электронный ресурс]. URL: https://github.com/vapermitin/genetic_algorithms_practice (дата обращения: 08.07.2026).